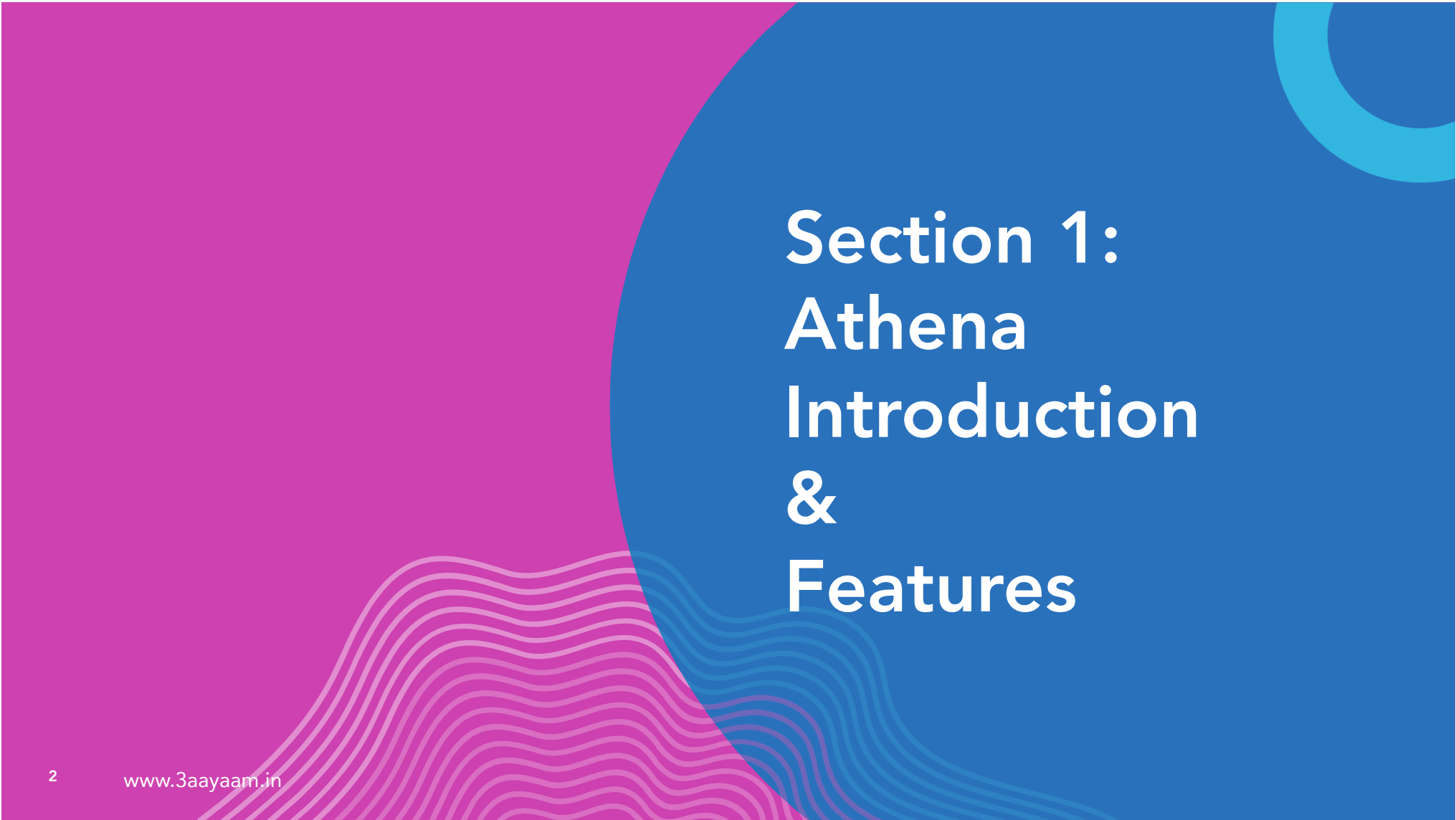


The background of the slide is composed of large, overlapping circles in shades of blue, purple, and magenta. At the bottom, there are wavy, concentric lines in green and yellow. In the top right corner, there is a grid of small white dots.

Athena

www.3aayaam.in



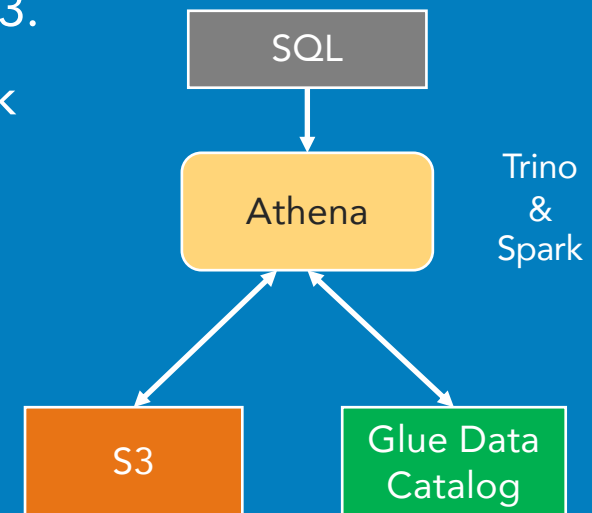
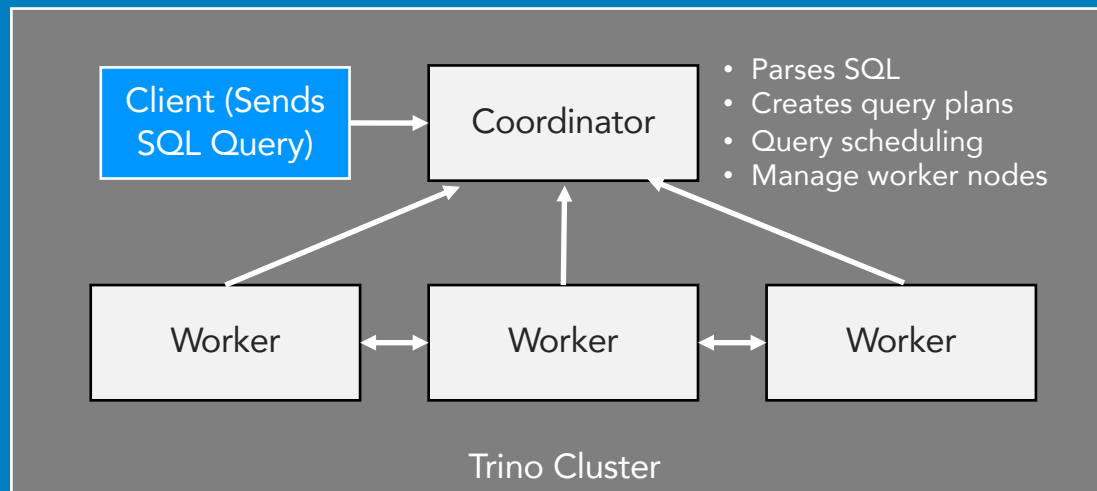
Section 1: Athena Introduction & Features

Agenda – Section 1

- Athena Introduction
- Use cases
- Databases & Tables
- Data Type, DDL, DML
- Athena SerDe
- Execution Parameters
- PREPARE & EXECUTE
- Schema Evolution
- Query JSON
- ACID
- Iceberg Tables
- Partitioning & Bucketing

Introduction

- SQL interactive query to analyze data that is in S3.
- Query Engines – Trino (written in Java) and Spark (written in Scala).
- Architecture:

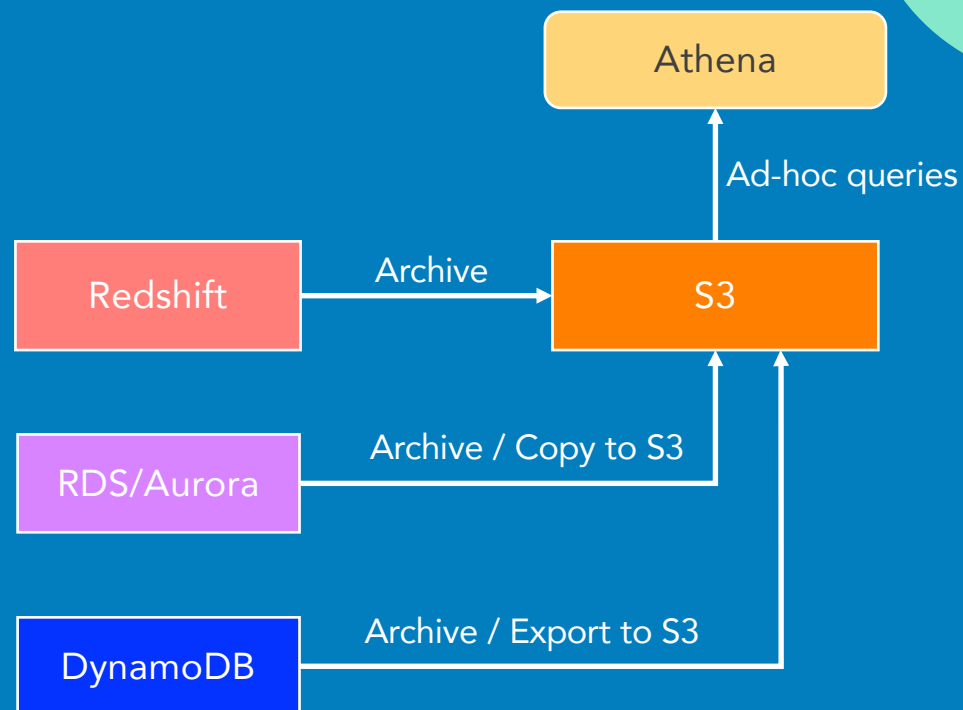


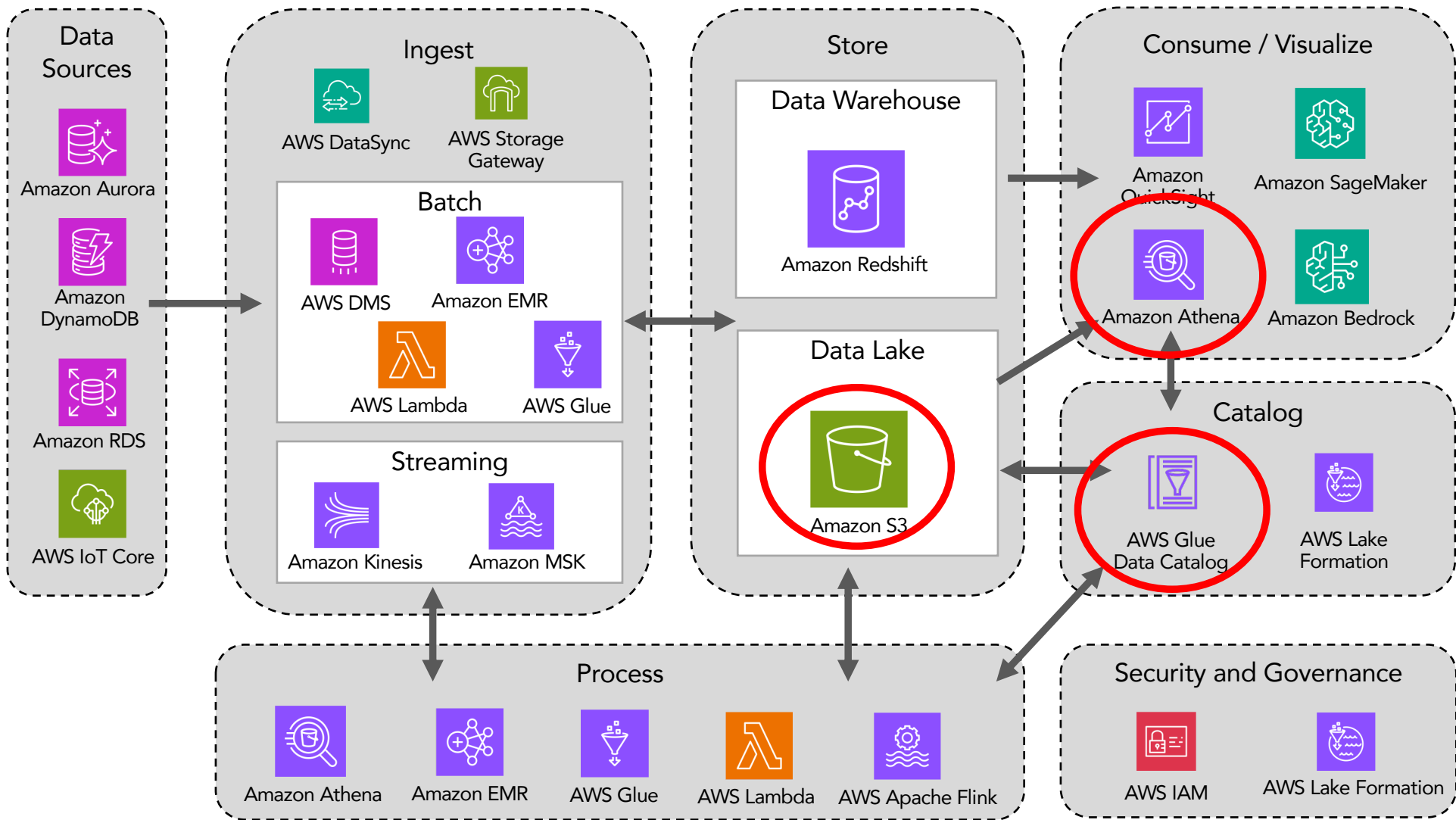
Introduction

- File formats:
 - CSV
 - JSON
 - Parquet
 - ORC
- Works with Glue Data Catalog and Lake Formation.
- Can work with Hive Metastore.
- Used for Ad-hoc analytics.

Use Cases

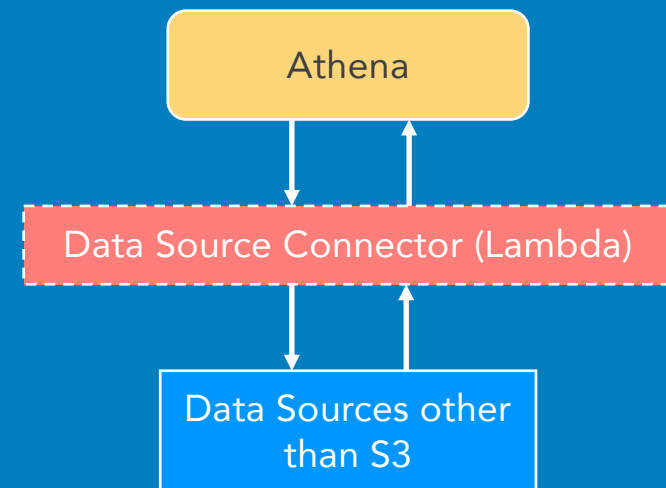
- When to use Athena??
- Ad-hoc analytics queries.
 - Not for complex queries with large no. of tables and datasets.
- Redshift archival.
- OLTP database archival.
- Redshift Spectrum? Can't it be used?
- Serverless.
- Explore console.





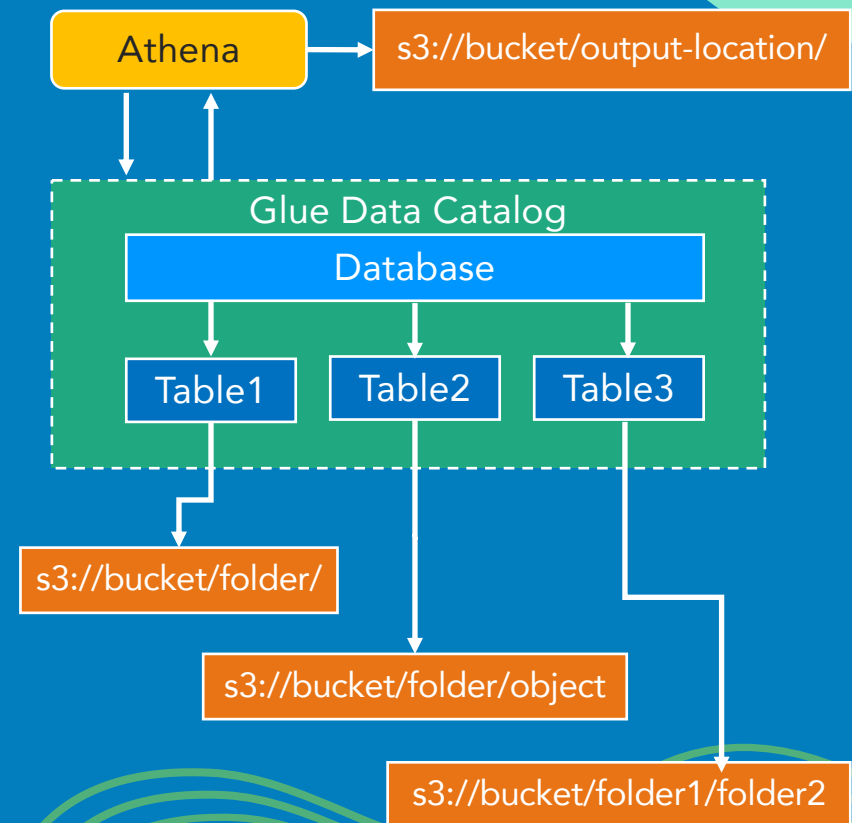
Federated Queries

- SQL on Anywhere!
- Federated queries are used on data sources other than S3.
- Data source connectors are used, translates between Athena and Data Sources.
- Data source connectors run on Lambda.
- AWS – Redshift, DocumentDB, DynamoDB, RDS, Aurora.
- Non-AWS – Azure Synapse & Data Lake Storage, BigQuery, GCS, HBase, Db2, Kafka, Redis OSS, Snowflake, Teradata etc.



Data Source – Databases & Tables

- Databases and tables are created in Glue Data Catalog using Athena SQL.
- Athena is used to query these objects in Glue DC.
- Query output location in S3.



Hands-on

- Use files from athena_dataset folder.
- Use Glue Crawler.
- Create database 3aayaam_athena_db_gdc.
- Create tables for the following:
 - Customers
 - Order_items
 - Orders
 - Sellers
- Set query output location in S3.

Data Types

- TINYINT / SMALLINT / INT / BIGINT
- FLOAT / DOUBLE / DECIMAL(p,s)
- CHAR / VARCHAR / STRING
- TIME / DATE / TIMESTAMP
- ARRAY / MAP / **STRUCT**

DDL Statements

- Create Database
- Create Table
- CTAS
- Create View
- Alter / Drop Table
- Alter Database
- Drop Database
- SHOW commands

```
CREATE DATABASE <database_name>  
LOCATION 's3://<bucket>';
```

```
CREATE EXTERNAL TABLE <db_name>.<table_name>  
(<col_name> data_type, ... )  
PARTITIONED BY (<col_name> data_type...)  
CLUSTERED BY (<col_name>, ...) INTO <num_buckets> BUCKETS  
ROW FORMAT <row_format>  
STORED AS <file_format>  
WITH SERDEPROPERTIES (...)  
LOCATION 's3://<bucket>/<folder>/';
```

```
CREATE TABLE <table_name>  
AS <SQL query>;
```

```
CREATE VIEW <view_name>  
AS <SQL query>;
```

Row & File Format

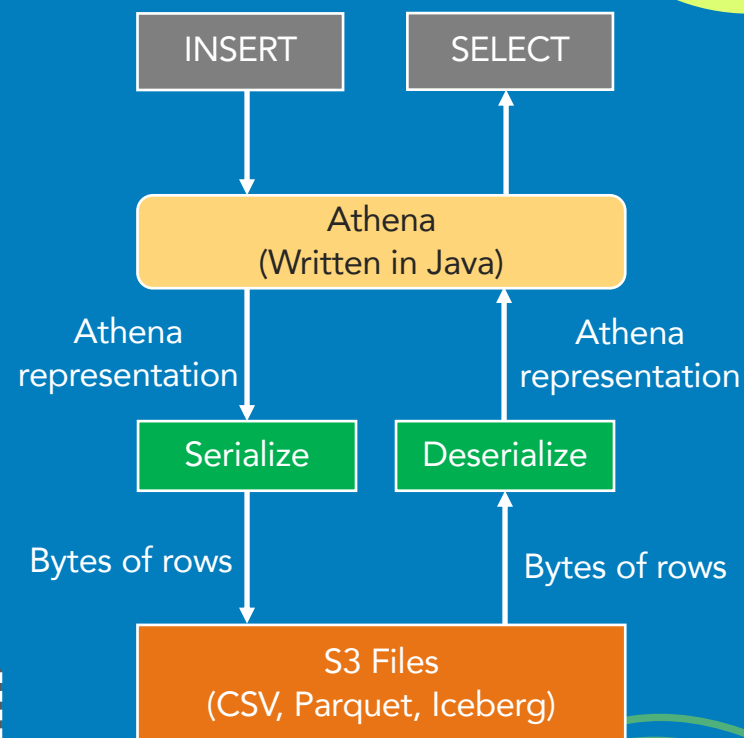
- Row Format
 - DELIMITED FIELDS TERMINATED BY 'char'.
 - LINES TERMINATED BY char.
- File Format
 - TEXTFILE – For CSV & TSV files.
 - ORC – For ORC files in S3.
 - PARQUET – For Parquet files in S3.
 - AVRO – For Avro files in S3.
 - JSON

SerDe

- What is SerDe (Serialize/Deserialize)
- Data Format and SerDe:

Data Format	SerDe Library (ROW FORMAT SERDE)
Avro	org.apache.hadoop.hive.serde2.avro.AvroSerDe
Parquet	org.apache.hadoop.hive ql.io.parquet.serde.ParquetHiveSerDe
CSV	org.apache.hadoop.hive.serde2.OpenCSVSerde
JSON	org.apache.hive.hcatalog.data.JsonSerDe
ORC	org.apache.hadoop.hive ql.io.orc.OrcSerde
Regex	org.apache.hadoop.hive.serde2.RegexSerDe

`ROW FORMAT SERDE '<serde-library-mentioned-above>'`



SerDe

- What is SerDe (Serialize/Deserialize)
- Data Format and SerDe:

Data Format	SerDe Library (ROW FORMAT SERDE)	File Format (STORED AS)
Avro	org.apache.hadoop.hive.serde2.avro.AvroSerDe	STORED AS AVRO
Parquet	org.apache.hadoop.hive ql.io.parquet.serde.ParquetHiveSerDe	STORED AS PARQUET
CSV	org.apache.hadoop.hive.serde2.OpenCSVSerde	STORED AS TEXTFILE
JSON	org.apache.hive.hcatalog.data.JsonSerDe	STORED AS JSON
ORC	org.apache.hadoop.hive ql.io.orc.OrcSerde	STORED AS ORC

STORED AS <file_format>

DML Statements

- SELECT
- INSERT INTO
 - Insert rows based on SELECT output.
 - Source and Target table formats can be different.
 - Formats – Parquet, CSV/ TSV (Textfile), Avro, JSON, ORC.
 - Files are written to data locations in S3.
- DELETE – Delete from Iceberg table.
- UPDATE – Update rows in Iceberg table.
- MERGE – Conditional update, delete, insert in Iceberg table.

```
SELECT
FROM <table1>, <table2>, ...
WHERE condition
GROUP BY ...
HAVING
UNION | INTERSECT | EXCEPT
ORDER BY ...
LIMIT
```

```
DELETE FROM <table> WHERE <condition>
```

```
UPDATE <table>
SET ...
WHERE <condition>
```

```
MERGE INTO <tgt_table>
USING <src_table>
ON <condition>
WHEN MATCHED THEN <sql-statement>
WHEN NOT MATCHED THEN <sql-statement>
```

Hands-on

- Drop database 3aayaam_athena_db_gdc and all its tables.
- Use Athena SQL.
- Create database 3aayaam_athena_db.
- Create tables for the following:
 - customers (Use SerDe for CSV)
 - order_items (Use SerDe for CSV)
 - orders (Use SerDe for Parquet)
 - sellers (Use SerDe for Avro)
- Run complex query joining orders, sellers, order_items and customers.
- Use CTAS to change file format

Query Execution Parameters

- What are parameters and its values?
- Place “?” in the parameter values.
- SELECT, INSERT INTO, CTAS, UNLOAD commands.
- Prepare and Execute statements also support query parameters.

```
SELECT order_id, customer_id, order_date
FROM orders
WHERE customer_id = '123abc'
AND payment_method = 'AMEX';
```

```
SELECT order_id, customer_id, order_date
FROM orders
WHERE customer_id = ?
AND payment_method = ?;
```

UNLOAD Command

- Extract query results from SELECT statement in a specified format.
- Parameters
 - format, compression
 - compression_level, field_delimiter

```
UNLOAD
(SELECT order_id, customer_id, seller_id, order_date, payment_method
 FROM orders)
TO 's3://<bucket>/<folder>/'
WITH
(
  format = 'PARQUET',
  compression = 'expression',
  field_delimiter = ',',
);
```

Prepare & Execute

- Creates SQL statement to be run at a later time.
- Query parameters can be included in the statement.

```
PREPARE statement_name FROM <sql query>;
```

- SQL Query – SELECT, CTAS, INSERT INTO. Query parameters (?) can also be used.
- Execute the statement.

```
EXECUTE statement_name USING parameter1, parameter2, parameter3;
```

Schema Evolution

- Handle schema updates.
- Schema-on-read – Schema is applied when data is read.
- Supported schema updates:

Schema Update	CSV	JSON	PARQUET	AVRO	ORC
Rename Columns	Y	N	N	N	N
Add columns at the beginning or middle of the table	N	Y	Y	Y	Y
Add columns at the end of the table	Y	Y	Y	Y	Y
Remove columns	N	Y	Y	Y	Y
Data type change	Y	Y	Y	Y	Y
Change column position	N	Y	Y	Y	Y

Query JSON

- Create table needs special data type – STRUCT.
- Query using DOT notation.
- Arrays using [n].
- Get "addr1" and "addr2"
 - **SELECT address.addr1, address.addr2**
- Get 2nd phone number
 - **SELECT address.phone[2]**

```
{  
  "name" : "3aayaam",  
  "address" : {  
    "addr1" : "Bangalore",  
    "addr2" : "India"  
    "phone" : [ "1234", "5678" ]  
  }  
}
```

Single Record

address : JSON

phone : ARRAY

Hands-on

- Use parameters to execute query on "sellers" table.
- Extract records from "orders" table for payment method of "MasterCard" and store the output in CSV format.
- Extract records from "sellers" table for country "India" and store the output in ORC format.
- Prepare an SQL statement on "orders" table to execute it later for different customers.
- Query "orders_json" table for orders that were placed on 2025 and customers with country "United States".

Hands-on – Schema Evolution

- Schema Evolution (CSV, Parquet, Avro).
 - Create table order_items with order_id, product_id.
 - Add order_date.
 - Change column position for product_id.
 - Keep both the files in one place and check behavior.
 - DROP COLUMN is only supported in Iceberg.
- JSON as String
 - JSON_EXTRACT
 - JSON_EXTRACT_SCALAR

order_id	product_id

schema1/

order_id	product_id	order_date

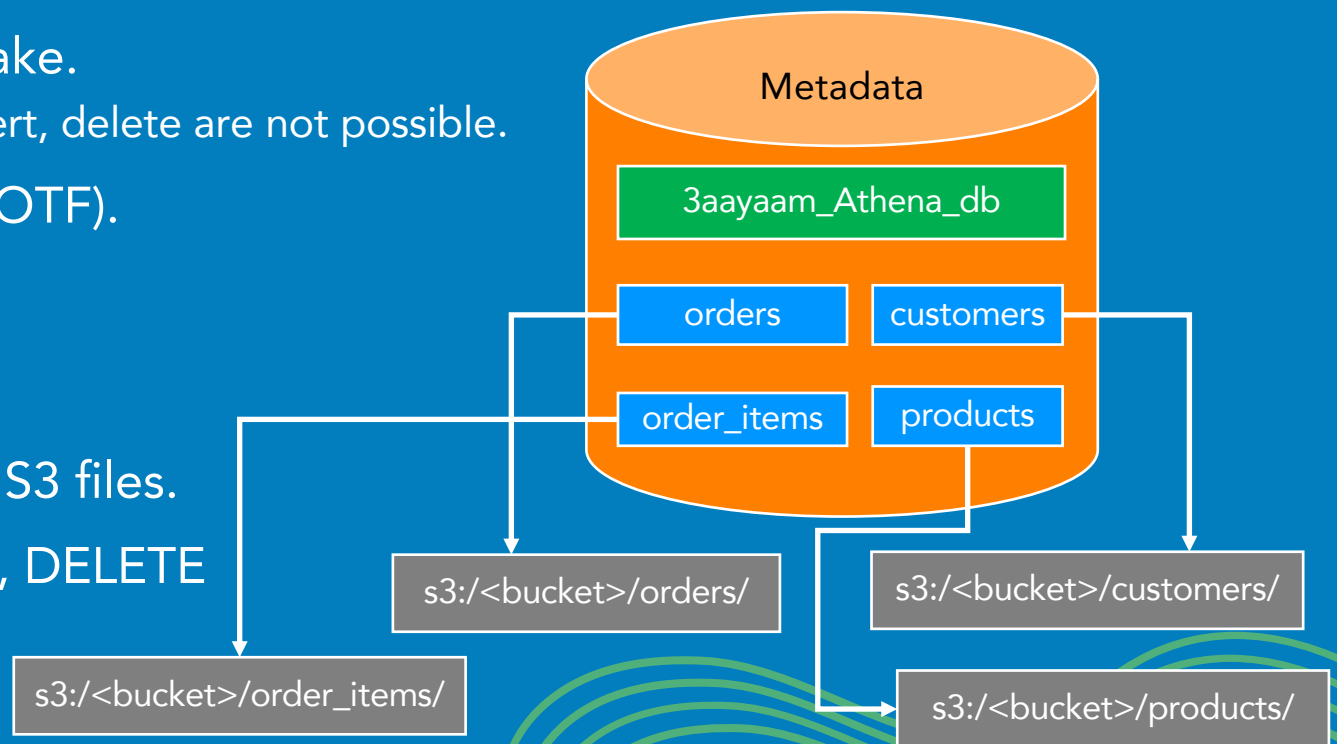
schema2/

order_id	order_date	product_id

schema3/

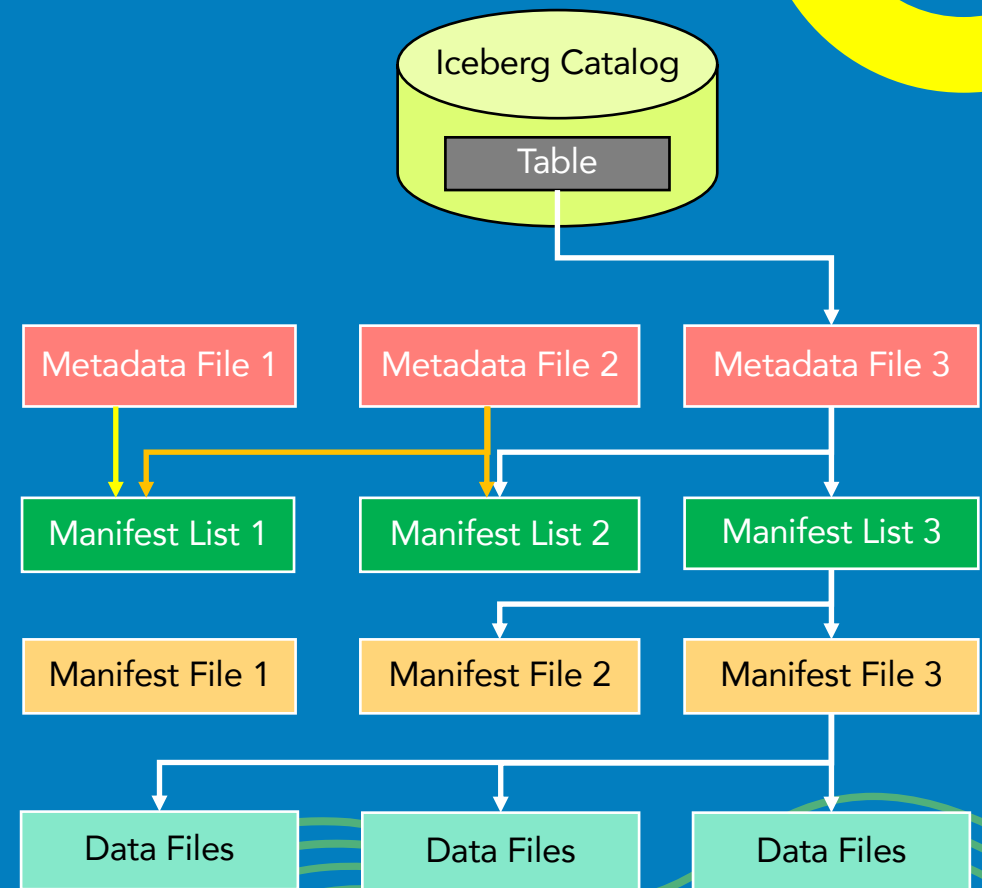
Lakehouse

- Data Lake – S3.
- Limitations of Data Lake.
 - Single row update, insert, delete are not possible.
- Open Table Format (OTF).
 - Iceberg
 - HUDI
 - Delta Lake
- Glue Data Catalog & S3 files.
- No INSERT, UPDATE, DELETE

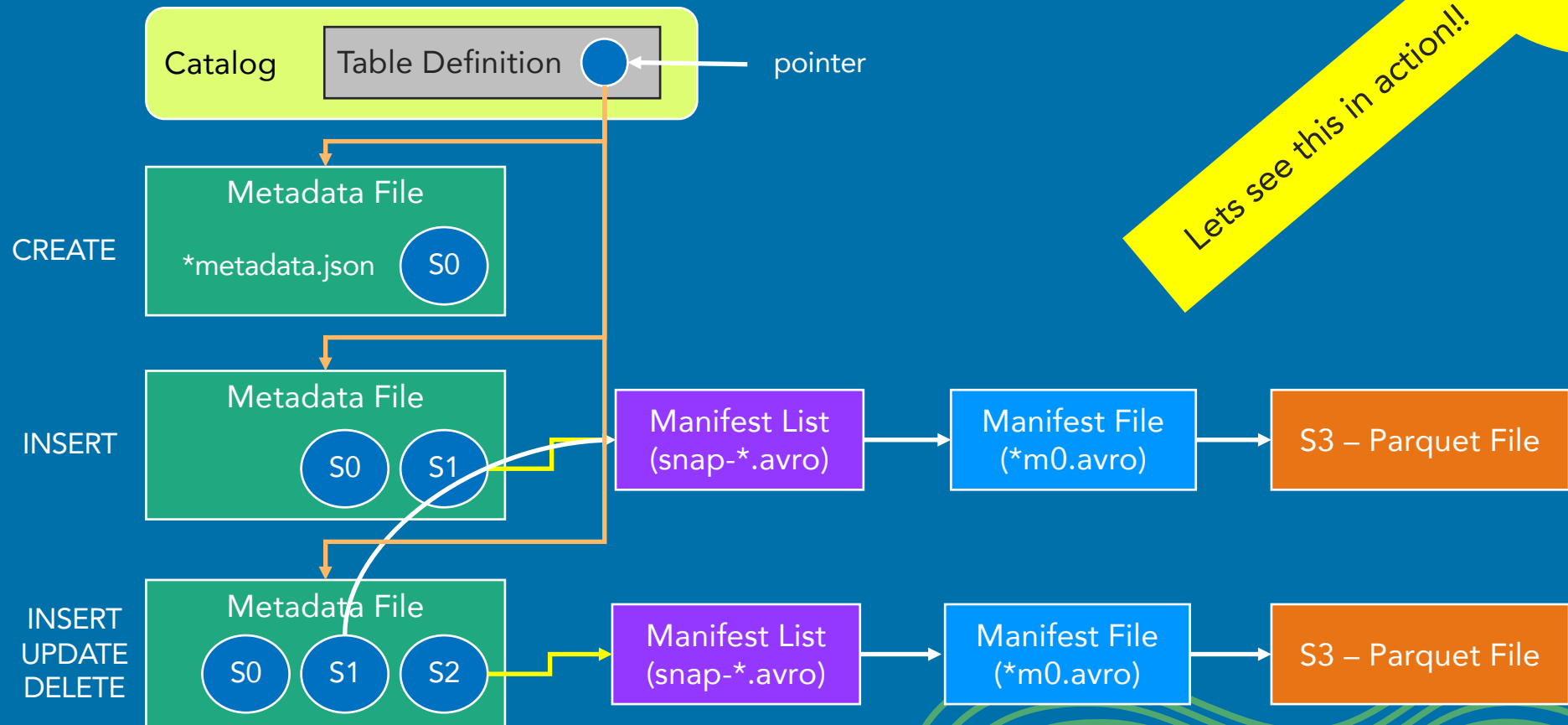


Iceberg Tables

- Provides metadata structure to enable row level changes in Data Lake.
- Architecture:
 - Iceberg Catalog
 - Metadata File – Table's schema, partition and snapshots. *.metadata.json
 - Manifest List – Information about manifest files. snap-*.avro
 - Manifest File – Information about data files. *m0.avro
 - Data File – Contains data along with delete files.

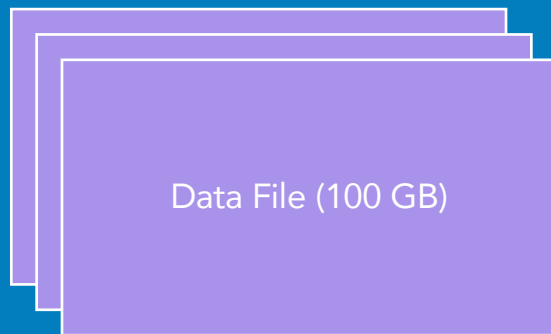


Architecture

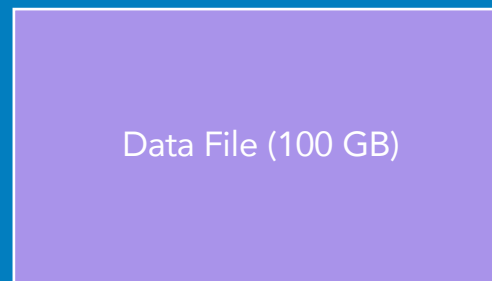


Delete Files

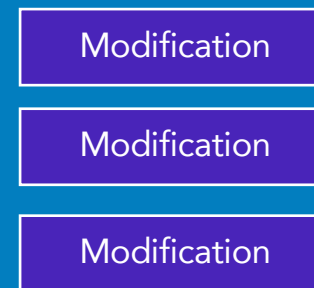
- Delete files – Files that contain only the changes.
- Copy-On-Write (COW) & Merge-On-Read (MOR)



COW



MOR



ACID

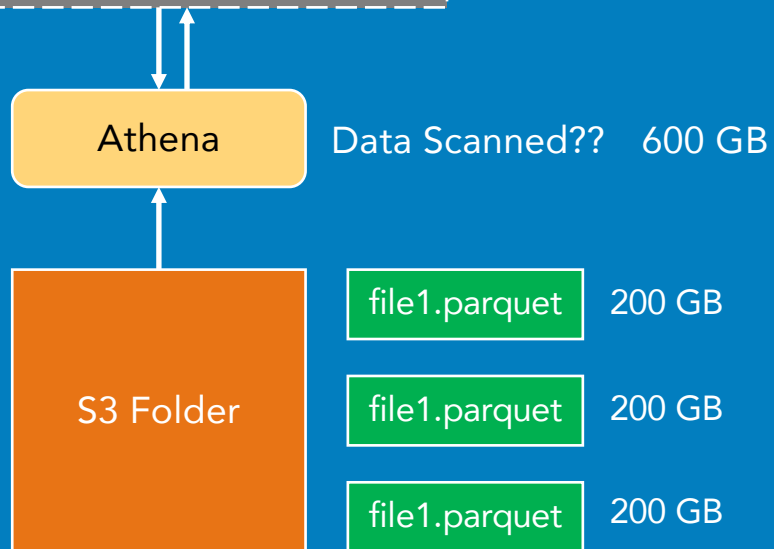
- Atomicity, Consistency, Isolation, Durability.
- Row level modifications.
- Concurrent users can write record.

OPTIMIZE and VACUUM

- Optimizes rows in an Iceberg table by rewriting data files into a more optimized layout based on the size and number of associated delete files.
 - OPTIMIZE order_items_iceberg
- Vacuum expires snapshots and removes unused files. Reduces storage consumption.
 - VACUUM order_items_iceberg

Partitioning

```
SELECT customer_id, name,  
... FROM customers  
WHERE country = 'India';
```



- Partitioning and bucketing are used to reduce the amount of data scanned.

Partitioning

```
SELECT customer_id, name,  
... FROM customers  
WHERE country = 'India';
```

Athena

Data Scanned??

150 GB

S3 Folder

country=India

150 GB

country=France

150 GB

country=USA

150 GB

country=Russia

150 GB

```
CREATE EXTERNAL TABLE customers (...)  
PARTITIONED BY (country varchar(40))  
STORED AS parquet  
LOCATION 's3://<bucket>/customers';
```

```
MSCK REPAIR TABLE customers;
```

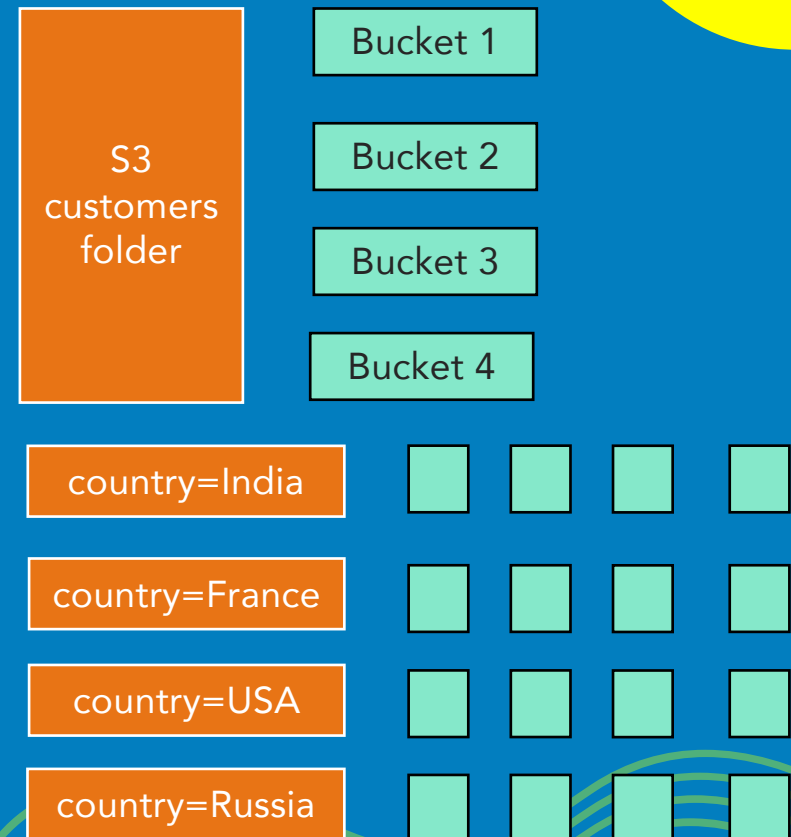
```
ALTER TABLE customers  
ADD PARTITION (country = 'UAE');
```

```
MSCK REPAIR TABLE customers;
```

Bucketing

```
CREATE EXTERNAL TABLE customers (...)  
...  
CLUSTERED BY ('customer_id') INTO 4 BUCKETS  
...  
TBLPROPERTIES (  
  'bucketing_format' = 'spark'  
)
```

- Further optimization by reducing data scan.
- Created automatically by Athena.
- Need to provide bucket key in WHERE clause.
- Partitioning and bucketing can be used together.



Hands-on

- Create partitioned table using country-wise files.
- DESCRIBE
- SHOW CREATE TABLE
- SHOW TABLES
- SHOW DATABASES
- SHOW COLUMNS
- SHOW PARTITIONS
- SHOW TBLPROPERTIES

Summary – Section 1

- Athena. Trino architecture. Use cases of Athena.
- Databases, tables, data types, DDL, DML statements.
- Row and File formats.
- SerDe.
- Execution parameters. Prepare & Execute commands.
- UNLOAD command to extract.
- Schema evolution.
- Iceberg with Athena. ACID.
- Partitioning and Bucketing.
- Optimize and Vacuum commands

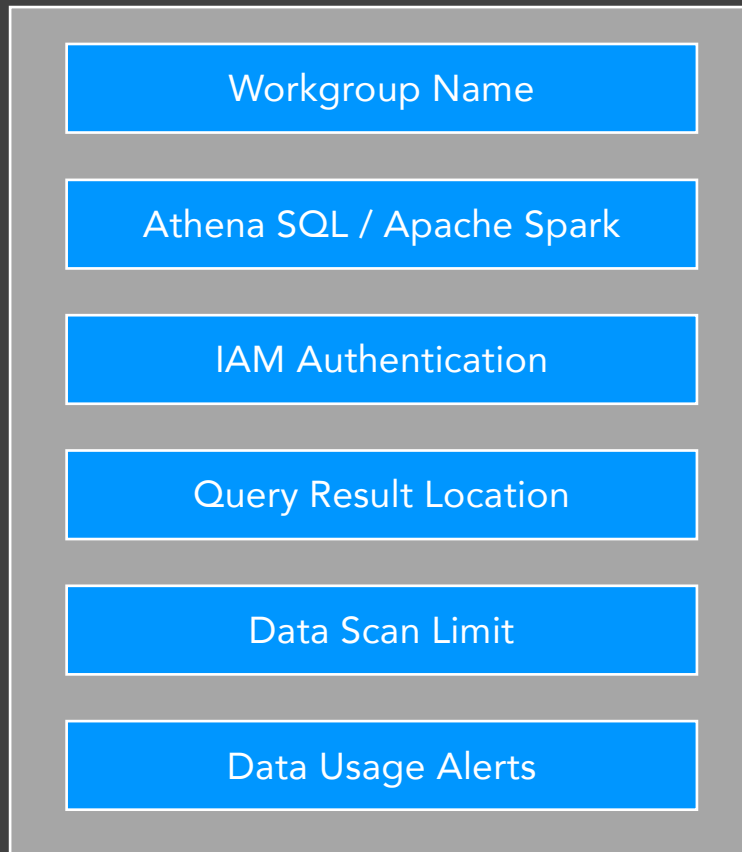


Section 2: Athena Workload Management (WLM)

Workload Management in Athena

- Workload separation.
- Access control.
- Capacity management.
- Configuration management.

Workgroups



Workgroup Name

Athena SQL / Apache Spark

IAM Authentication

Query Result Location

Data Scan Limit

Data Usage Alerts

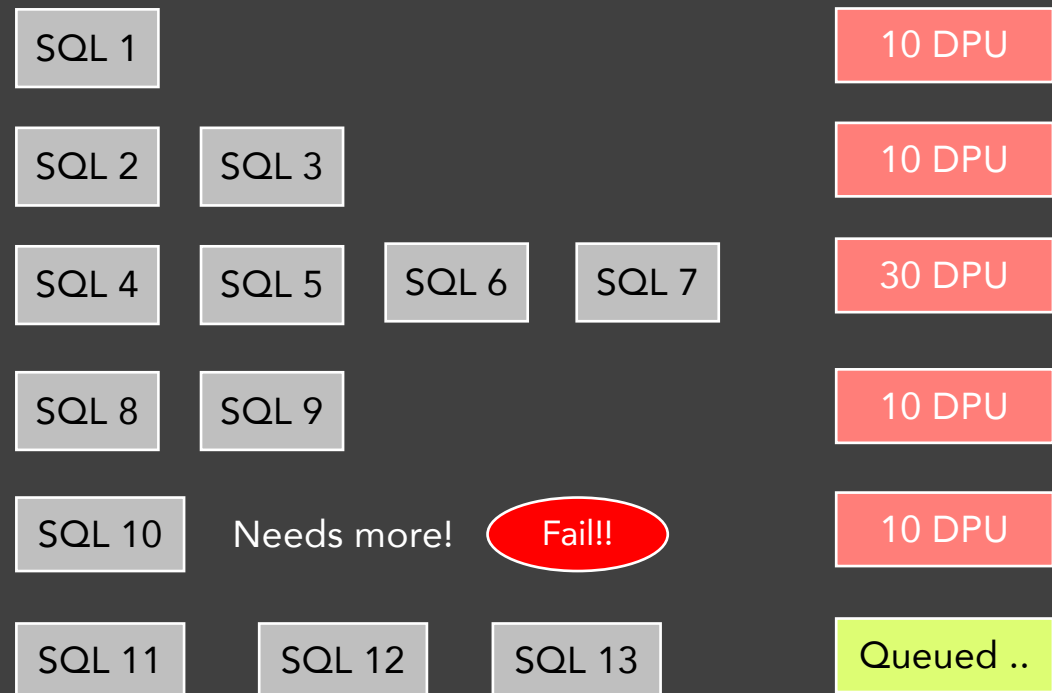
- Query Engine – Trino or Spark?
- Use access using IAM.
- Where query output will be stored.
- Limit for the maximum amount of data a query is allowed to scan. Applies to all queries in the workgroup.
- Send alerts if queries in the workgroup scan specified amount of data for specified period.
- Workgroup can be specified at query level using the console, API, JDBC/ODBC.

Query Processing Capacity

- Handled by capacity reservations to scale workloads, when queries need dedicated compute resources.
- Used to enable concurrent query runs.
- Attached to workgroups.
- Capacity is measured in DPUs.
 - 1 DPU = 4 vCPUs and 16 GB RAM.
 - Soft limit of 1000 DPUs per account per region.
 - DDLs take 4 DPUs. DMLs can use 4 to 124 DPUs.
- DPU requirement is handled when a query is submitted.
- No. of DPUs for DMS – Size of data, File format, Query complexity etc.

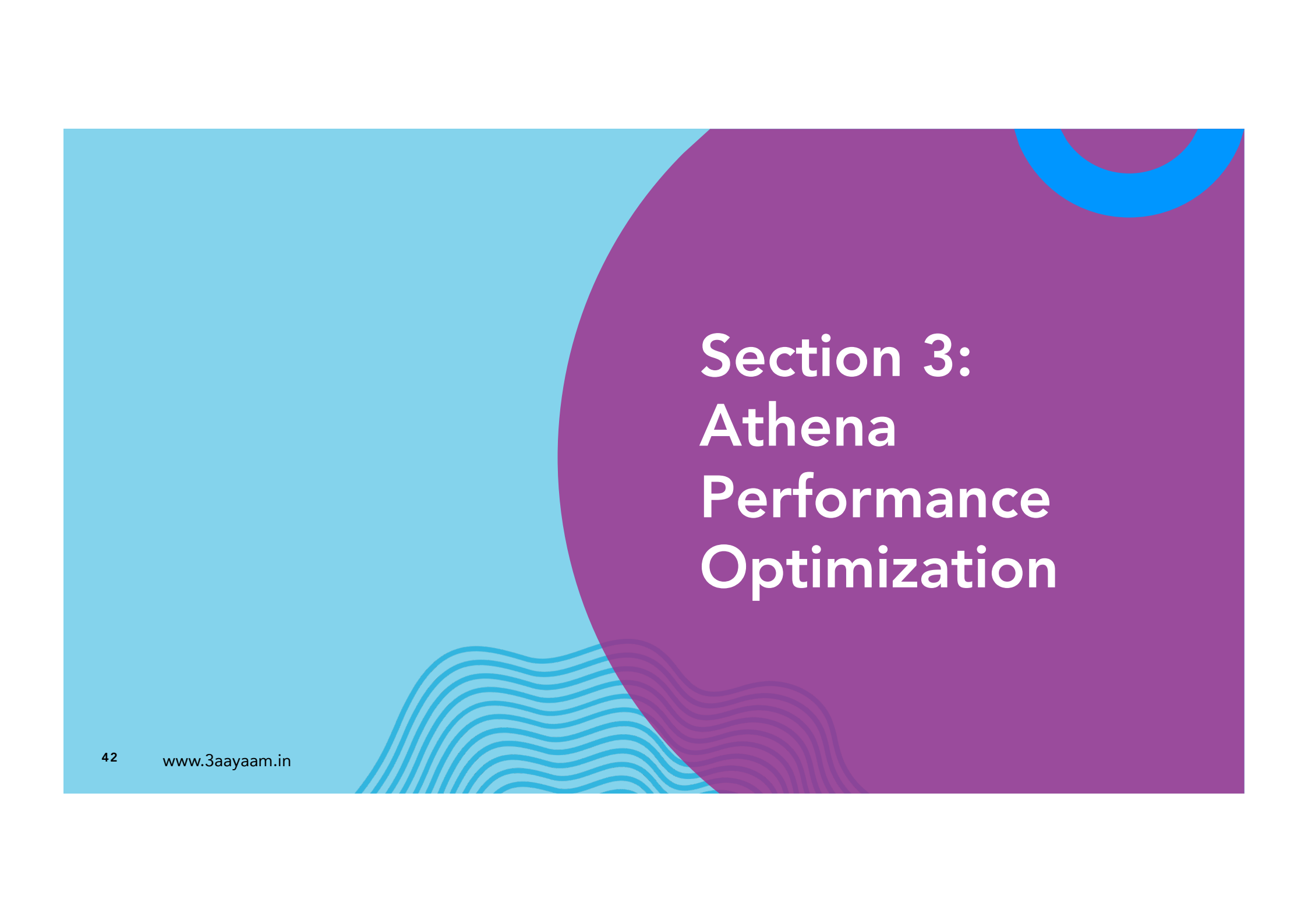
Query Processing Capacity

- Capacity consumption.
- 70 DPUs provisioned.
- CloudWatch metrics:
 - DPU Allocated.
 - DPU Consumed – for actively running queries.
 - DPU Count – Max DPUs consumed by query once it is complete.
 - Query Queue Time – Wait time of a query for DPUs/resources.



Hands-on

- Create workgroup ("lookup-queries") and limit data scan to 5 GB and execute a complex query.
- Create 2nd workgroup ("ad-hoc_analytics") and limit data scan to 500 GB and execute the same query.
- Use capacity reservation with 25 DPUs and attach them to 2nd workgroup. Run some queries and check CloudWatch metrics.

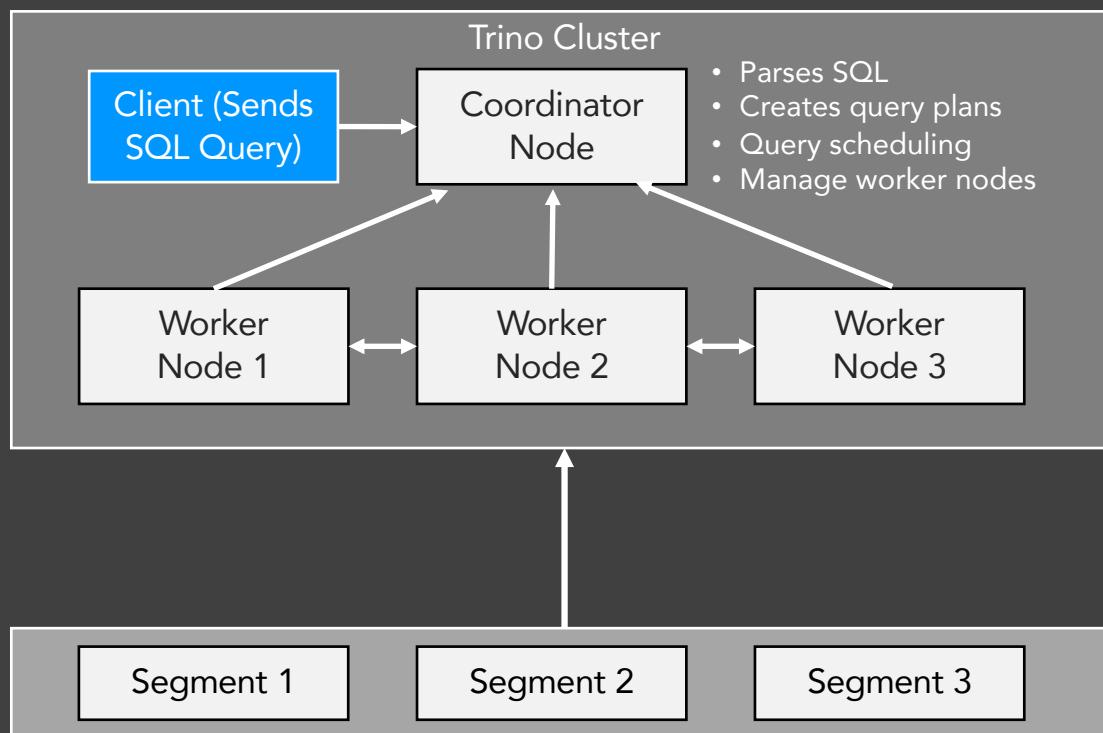


Section 3: Athena Performance Optimization

Storage Level Optimization

- Use columnar file formats – Parquet, ORC.
- Use Partitioning. Partition column has to be used in the WHERE clause.
- Use Bucketing. Clustered column has to be used in the WHERE clause.
- Avoid too many files in a folder.

Architecture



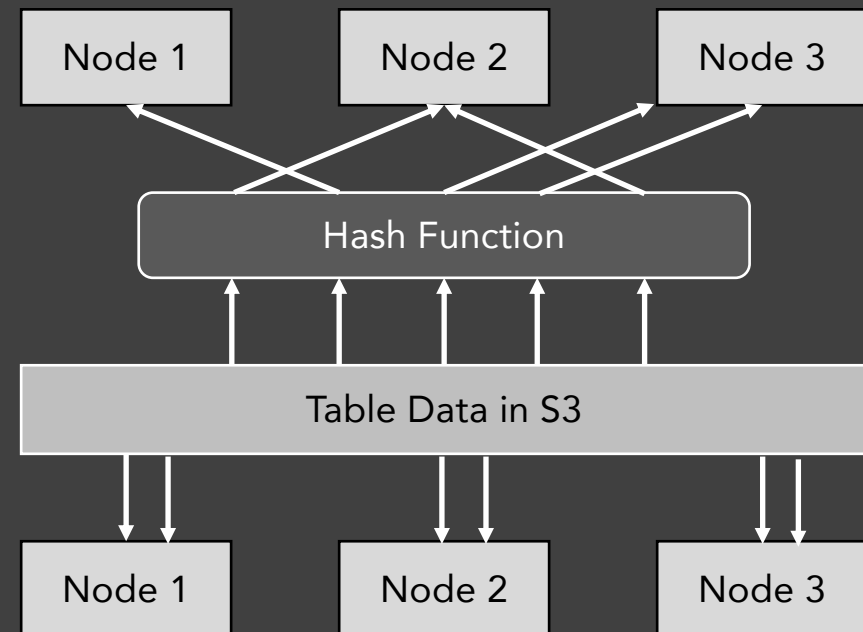
- Processing Stages.
- Exchange of data across nodes.
- Different join types.
- Data Scanning.

EXPLAIN Plan

- **EXPLAIN** <query>
- **EXPLAIN ANALYZE** <query>
- Output format (FORMAT) – TEXT & JSON.
- TYPE – LOGICAL, DISTRIBUTED, IO.
- Logical Plan – Logical plan of the SQL engine.
 - **EXPLAIN (TYPE LOGICAL)** <query>
- Distributed Plan – Execution plan in the distributed environment.
 - **EXPLAIN (TYPE DISTRIBUTED)** <query>

EXPLAIN Operators 1

- Query execution is divided into processing stages (fragments).
 - Single – The processing stage is performed on a single node.
 - Hash – Input data (of a table) is distributed across nodes using hash function and the stage is processed.
 - Round_robin – Input data (of a table) is distributed across nodes using round-robin fashion and the stage is processed.
 - Broadcast – The input data (of a table) is broadcasted to all nodes.



EXPLAIN Operators 2

- Data exchange – How data is moved across nodes.
 - LocalExchange – Data transfer happens within the node for different stages in a query.
 - RemoteExchange – Data transfer across nodes for different stages in a query.
- Logical Data Exchange – during logical plan.
 - Gather – Single node gathers output from all the nodes.
 - Repartition – Data is sent to specific node based on partition scheme.
 - Replicate – Data is copied to all nodes.
- Distributed Data Exchange – during distributed plan.
 - Hash – Data distribution across nodes using hash functions.
 - Single – Data distribution to a single node.

EXPLAIN Operators 3

- Scans:
 - TableScan – Scan table's data from S3.
 - ScanFilter – Scan and apply partition level scan and non-partitioned scan as per the WHERE clause.
 - ScanFilterProject – Does ScanFilter and projects columns that will improve performance in later stages.
- Joins: CrossJoin, InnerJoin, LeftJoin, RightJoin, FullJoin.
- Join distribution:
 - Partitioned – Both tables are hash-partitioned across nodes.
 - Replicated – One table is hash-partitioned across nodes and other table is replicated to all nodes.

Hands-on

- Create workgroup ("lookup-queries") and limit data scan to 5 GB and execute a complex query.
- Create 2nd workgroup ("ad-hoc_analytics") and limit data scan to 500 GB and execute the same query.
- Use capacity reservation with 25 DPUs and attach them to 2nd workgroup. Run some queries and check CloudWatch metrics.

Athena with Spark

- Uses Apache Spark as Query Engine.
- Spark applications for processing.

Athena Pricing (N. Virginia)

- Athena SQL - \$5 per TB of data scanned.
- Provisioned Capacity - \$0.30 per DPU hour.
- Apache Spark - \$0.35 per DPU hour.
- Example:
 - Approx. 2 TB of data is being archived from Redshift to S3 every month and kept in S3 for 6 months and then expired. 5 ad-hoc queries are run every week which reads approx. 8TB of data because of complex joins.
 - Dedicated compute capacity is needed for batch queries. 50 DPUs have been provisioned. 40-45 DPUs are used weekly for 1-2 queries that run for 8 hrs.

Athena Pricing (N. Virginia)

- Approx. 2 TB of data is being archived from Redshift to S3 every month and kept in S3 for 6 months and then expired. 5 ad-hoc queries are run every week which reads approx. 8TB of data because of complex joins.

	Data Stored (TB)	Data Scanned (TB)	Price (USD)
Month 1	2	32 TB	\$160.00
Month 2	4	64 TB	\$320.00
Month 3	6	96 TB	\$480.00
Month 4	8	128 TB	\$640.00
Month 5	10	160 TB	\$800.00
Month 6	12	192 TB	\$960.00
Month 7	12	192 TB	\$960.00
Month 8	12	192 TB	\$960.00

Athena Pricing (N. Virginia)

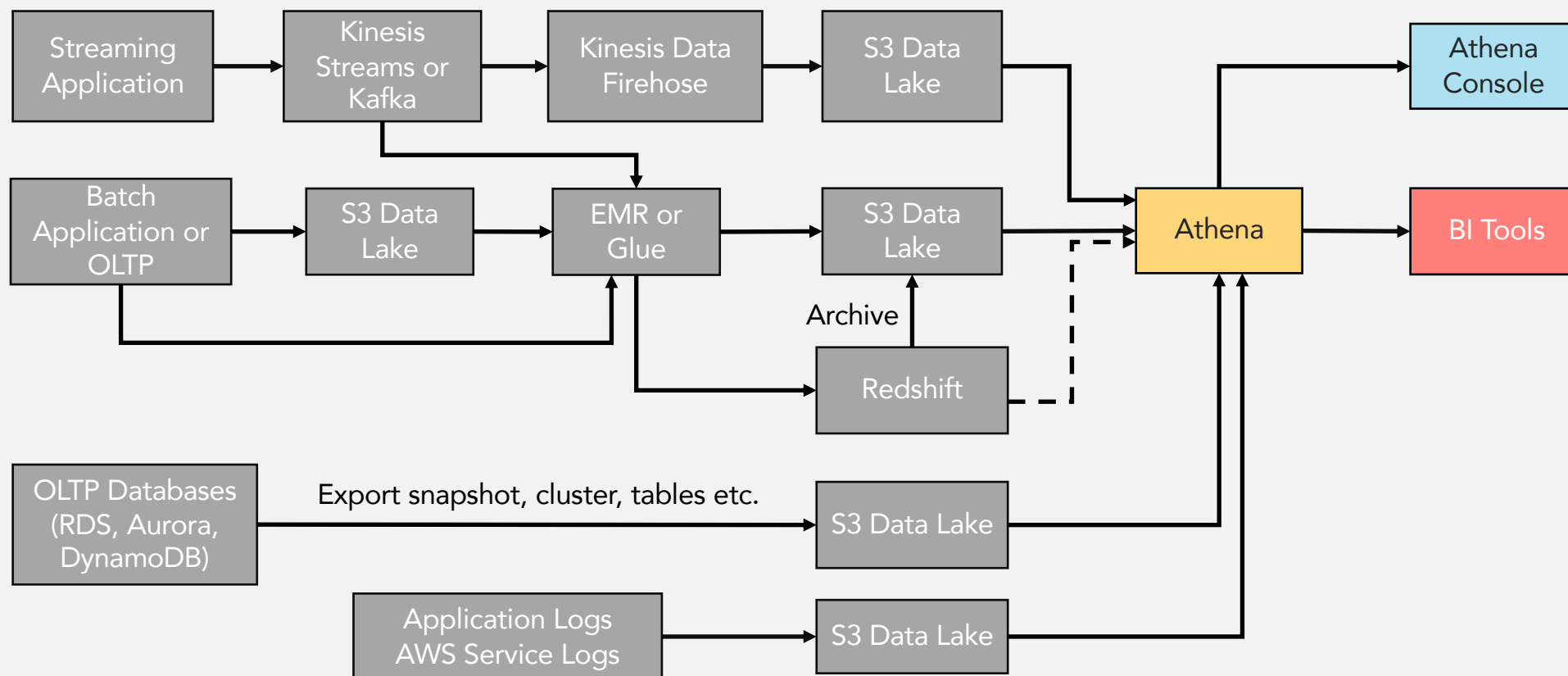
- Dedicated compute capacity is needed for batch queries. 50 DPUs have been provisioned. 40-45 DPUs are used weekly for 1-2 queries that run for 8 hrs.

	DPU's Used Monthly	Total Monthly Hours	Cost Calculation	Total Monthly Cost
Month 1	160	32	$160 \times 0.3 \times 32$	\$1536.00
Month 2	160	32	$160 \times 0.3 \times 32$	\$1536.00
Month 3	160	32	$160 \times 0.3 \times 32$	\$1536.00
Month 4	160	32	$160 \times 0.3 \times 32$	\$1536.00
Month 5	160	32	$160 \times 0.3 \times 32$	\$1536.00
Month 6	160	32	$160 \times 0.3 \times 32$	\$1536.00

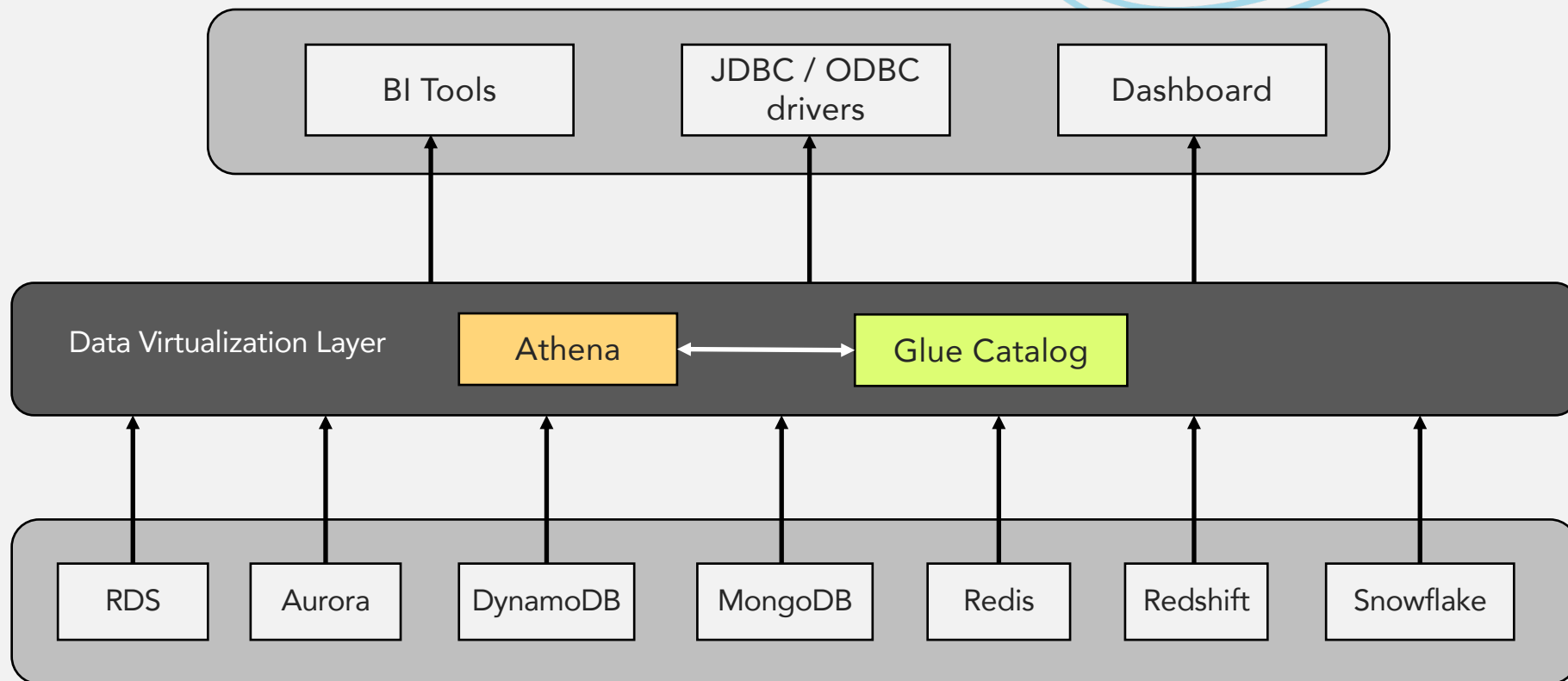
Summary – Section 2 & 3

- WLM in Athena using Workgroups.
- Data scanned, Provisioned capacity using DPU.
- Performance Optimization:
 - Storage level optimization.
 - EXPLAIN plan.
 - Explain operators.
- Athena with Spark.
- Pricing example.

Architecture using Athena



Architecture using Athena





Thank You!

www.3aayaam.in